

Movie Search na Spin/WASM vs OCI

Porównanie WebAssembly jako izolacji dla mikroservisów HTTP

44 471

filmów w fixture

1 core

wspólny Rust core

2 runtime'y

Spin/WASI i OCI

**Benchmark
mikroservisów
HTTP
bez udawania
zgodności
Meilisearch**

Pełny przebieg:
2026-05-24

Czy ten sam mikroservis działa praktycznie w Spin/WASI i OCI?

Najpierw dowodzimy zgodności funkcjonalnej, dopiero potem porównujemy cold start, load i pamięć.

- Ten sam kod aplikacyjny: movie-search-core
- Ten sam dataset: 44 471 filmów po deduplikacji id
- Te same endpointy HTTP i te same zapytania benchmarkowe
- Porównanie runtime/isolation, nie dwóch różnych wyszukiwarek

Do czego służy Spin/WASM/WASI/wasmtime?

WebAssembly

przenośny format binarny i sandbox wykonania

WASI

kontrolowane interfejsy: HTTP, filesystem, clock, random

wasmtime

runtime wykonujący komponenty poza przeglądarką

Spin

framework i CLI dla mikroservisów WebAssembly

OCI

standard obrazu i runtime kontenera jako baseline

Dlaczego odeszliśmy od pełnego Meilisearch?

Meilisearch był dobrym case study feasibility, ale złym benchmarkiem 1:1 dla Spin.

- Oficjalny Meilisearch opiera storage o LMDB i memory-mapped files
- Spin fallback miał inną semantykę rankingową i inny model danych
- Upstream WASI build zatrzymał się na zależnościach runtime/storage
- Wniosek: uczciwiej mierzyć własny wspólny serwis niż udawać parytet

Co robi movie-search?

Aplikacja jest prosta celowo: ma odsłonić koszt runtime'u, a nie imitować pełny silnik wyszukiwarki.

GET

/health · /version · /stats · /movies

POST

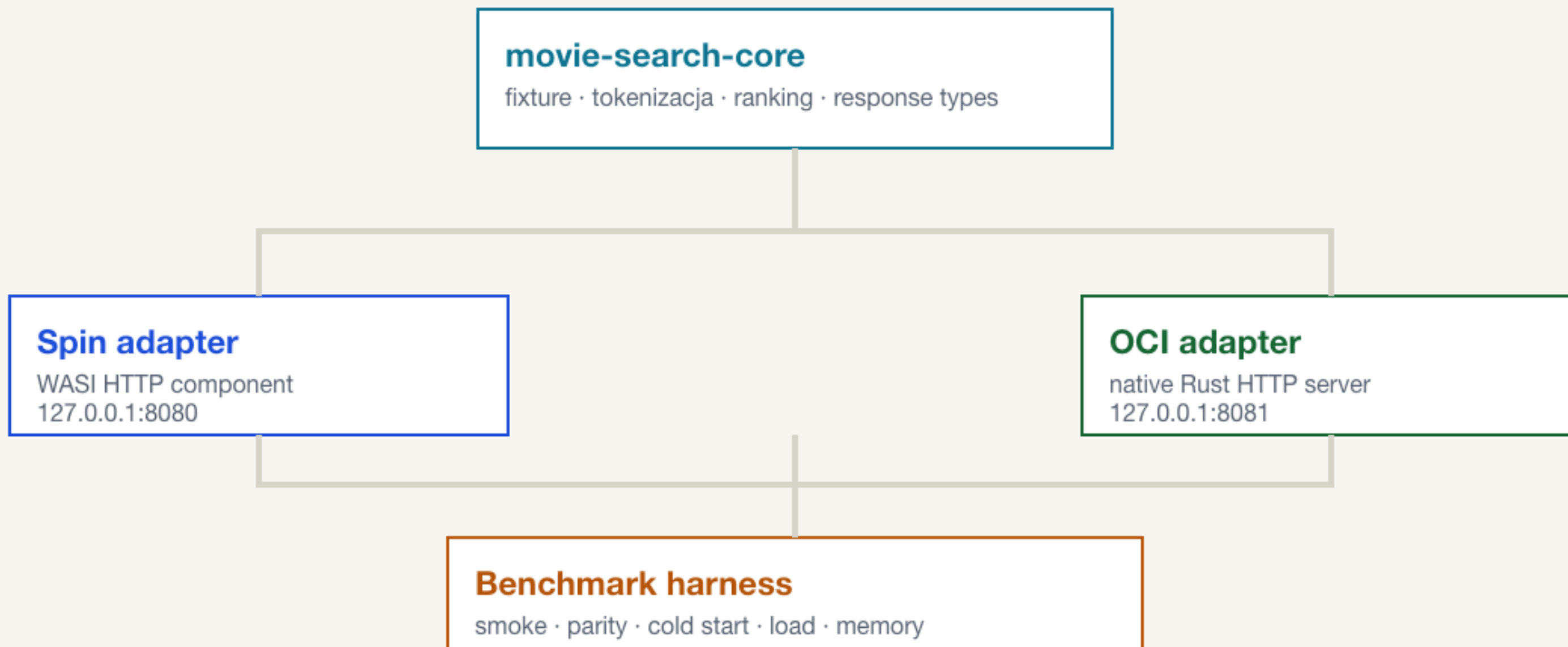
/search { q, offset, limit }

ranking

tokeny, pola, id rosnąco

- Deduplikacja id metodą last-write-wins
- Brak typo tolerance, fuzzy rankingu i reguł Meilisearch
- Puste zapytanie zwraca deterministyczną listę po id
- Zapytanie space skanuje pola title, genre i overview

Jeden core, dwa adaptery runtime



Jak mierzyliśmy?

1

Testy i build

cargo test, spin build,
docker compose build

2

Parytet

te same hit.id dla 5
zapytań

3

Cold start

20 powtórzeń na
runtime

4

Load

concurrency 10, 50,
100, 200

5

Memory

Docker stats vs host
process RSS

Pełny benchmark: benchmarks/run_all.sh. Demo pokazuje zgodność i artefakty, nie uruchamia pełnego testu obciążeniowego.

Oba runtime'y zwracają te same wyniki

0

błędów walidacji

44 471

documentCount

5

zapytań parity

'space': [62, 957, 1542, 2157, ...]

'toy story': [862, 863, 10193, ...]

'dark knight': [155, 29751, 49026, ...]

": [2, 3, 5, 6, 11, ...]

Spin and OCI movie-search results match.

Cold start: OCI szybciej gotowe, Spin szybciej kończy pierwszy search

28.4 ms

OCI p95 /health

144.2 ms

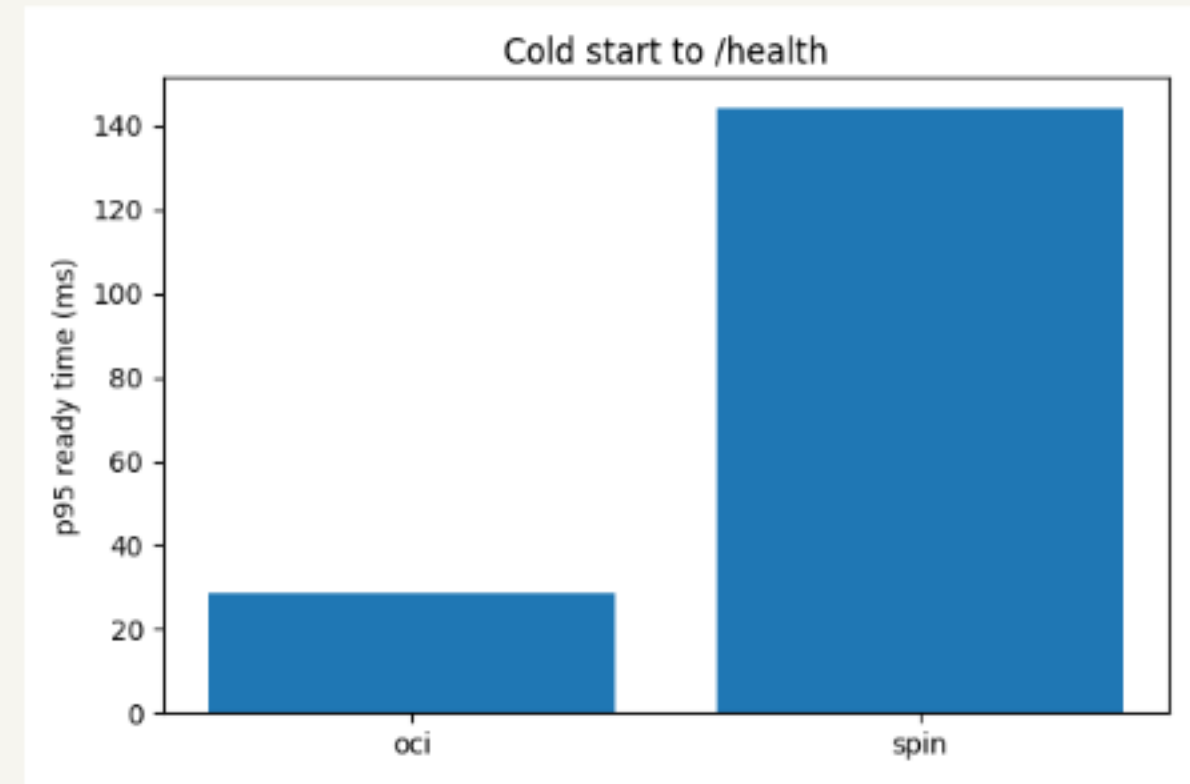
Spin p95 /health

377.0 ms

OCI p95 first search

217.5 ms

Spin p95 first search



Interpretacja: /health mierzy gotowość procesu, a pierwszy /search mierzy już realną ścieżkę aplikacji.

Zapytanie space: liniowe skanowanie obciąża oba runtime'y

77.2 req/s

Spin c=10

54.0 req/s

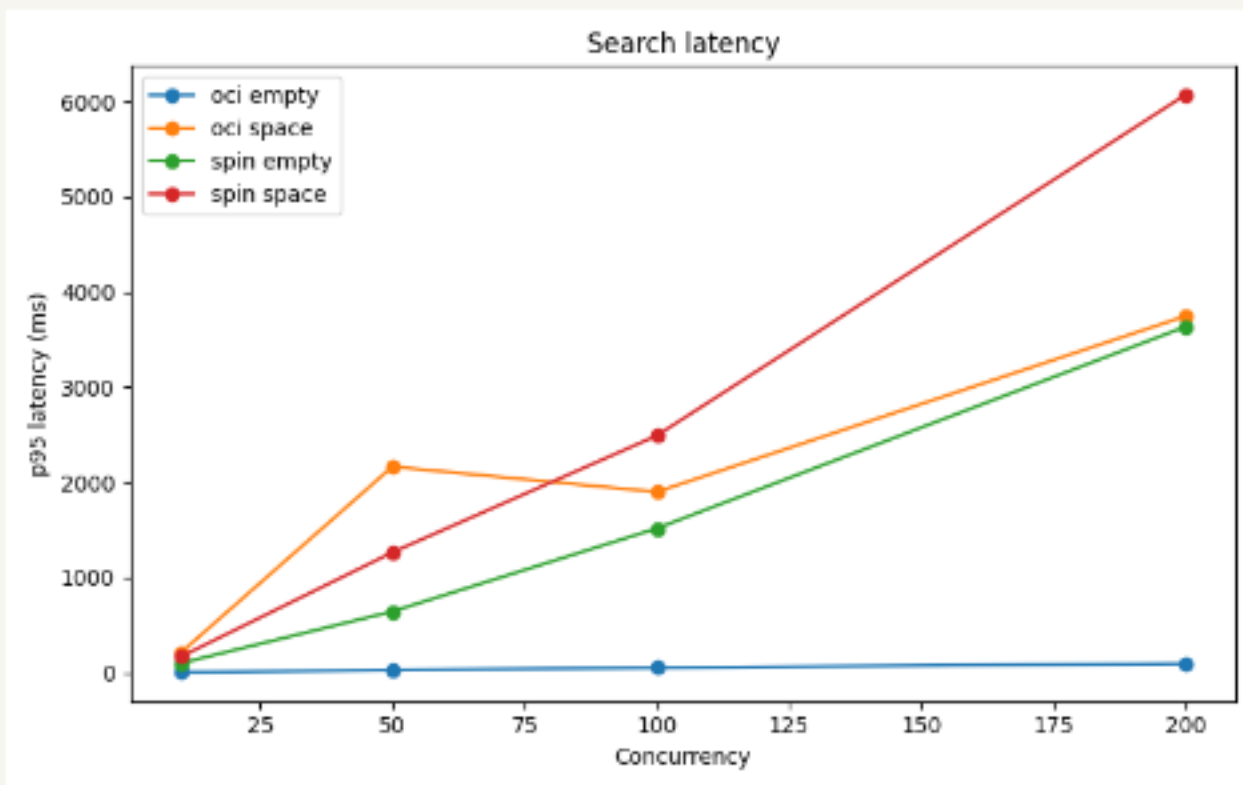
OCI c=10

168 ms

Spin p95 c=10

211 ms

OCI p95 c=10



Dla space oba warianty wykonują kosztowne liniowe skanowanie tekstu; przy wysokiej współbieżności rośnie tail latency.

Puste zapytanie pokazuje koszt ścieżki runtime/HTTP

3315 req/s

OCI c=10

133 req/s

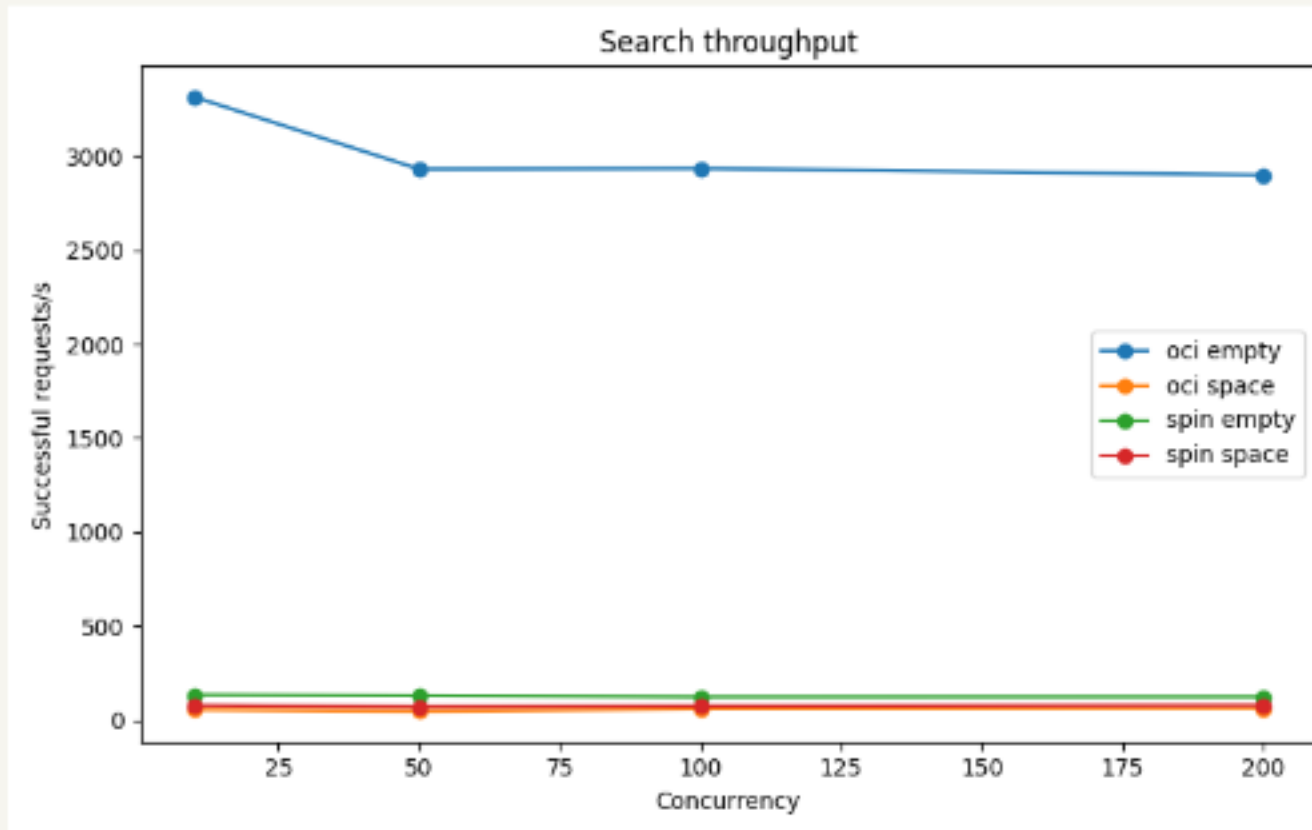
Spin c=10

95.7 ms

OCI p95 c=200

3636 ms

Spin p95 c=200



Puste zapytanie jest lekkie aplikacyjnie, więc mocno ujawnia narzut ścieżki HTTP/runtime.

Memory: metryki są porównywalne operacyjnie, ale nie identyczne

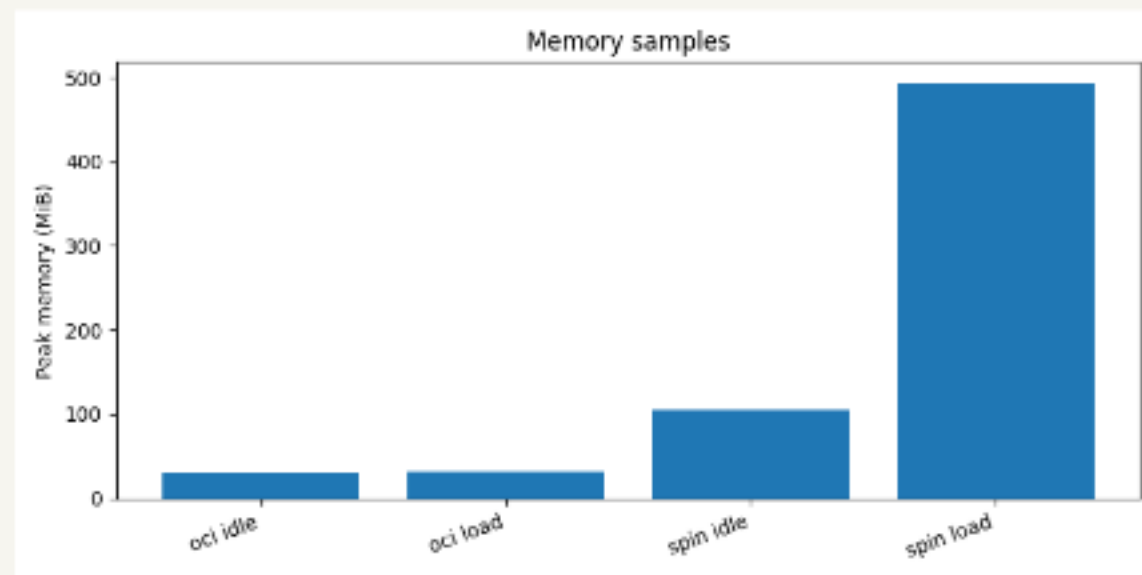
31.9 MiB

OCI load max

493.5 MiB

Spin load max RSS

- OCI: docker stats / cgroup view
- Spin: host process tree RSS
- Wniosek: porównanie operacyjne, nie identyczna rachunkowość



Demo: pokazujemy zgodność, nie odpalamy pełnego benchmarku na żywo

- Uruchom Spin :8080 i OCI :8081
- Pokaż /health, /version, /stats
- Wyślij POST /search dla q=space
- Uruchom benchmarks/compare_results.sh
- Pokaż CSV i wykresy z pełnego benchmarku

```
curl /version
curl /stats
curl -X POST /search
benchmarks/compare_results.
sh
ls results/plots
```

Pełny benchmark trwa kilka-kilkanaście minut, więc live demo pokazuje procedurę i dowód parytetu.

Kiedy ten stack jest sensowny?

Spin/WASM ma sens gdy

- mały izolowany HTTP component
- edge/serverless/plugin
- ważna przenośność i sandboxing

OCI pozostaje lepsze gdy

- potrzebna dojrzałość toolingowa
- storage-heavy service
- precyzyjna rachunkowość zasobów

Nasz wynik

- parytet funkcjonalny: tak
- wydajność: zależna od ścieżki
- Meilisearch port: feasibility blocker

Najważniejsze: benchmark jest uczciwy, bo porównuje ten sam kod aplikacyjny.